Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 5

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Model SwiftCargo's warehouses, transit hubs and delivery points as a directed multistage graph. Find the fastest or lowest fuel-cost route through every stage. Support traffic and weather delays, road closures and multiple delivery requests.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Analyze a dynamic-programming route algorithm and its computational cost.
- Find valid stage-by-stage routes and replan when road conditions change.

Theory

A multistage graph groups cities into ordered stages. Each directed road goes from stage s to stage s+1. Therefore, a delivery passes through every stage and the graph has no cycle. Dynamic programming solves later stages first and uses those answers to find the best route from earlier stages.

Fastest mode uses base travel time + traffic delay + weather delay. Cheapest mode uses estimated fuel litres x fuel price. Only one objective is minimized in each request. Closed roads are ignored; fuel cost excludes tolls and other expenses.

Working of the Algorithm

- 1. Set the chosen destination's cost to zero and all other costs to infinity.
- 2. Process stages backwards. For each open road, compare its weight plus the best remaining cost.
- 3. Save the best next road to print the full route. Report when the destination is unreachable.
- 4. Update road conditions by ID and process fresh requests; a batch uses the current fixed conditions.

Complexity Analysis

Each city and road is processed a constant number of times. With V cities and E roads, one request takes O(V + E). A batch of Q requests takes O(Q(V + E)). Road updates by known ID take O(1).

Time Complexity

Best Case: O(V + E) per request.

Average Case: O(V + E) per request.

Worst Case: O(V + E) per request; the complete stage scan is performed.

Space Complexity

O(V + E) for adjacency lists, stage buckets and route arrays. This avoids a V-by-V adjacency matrix.

Algorithm - Steps

- Step 1: Read the city stages, fuel price, roads and requested endpoints.
- Step 2: Choose fastest or cheapest mode and solve backwards through the stages.
- Step 3: Print the route, road IDs, travel time, fuel use and fuel cost.

Pseudocode

```
best[destination] = 0; all other best values = infinity
for stage from last-1 down to 1:
    for each city u in stage:
        for each open road (u,v):
            if weight(u,v) + best[v] < best[u]:
                best[u] = weight(u,v) + best[v]
                nextRoad[u] = (u,v)
follow nextRoad from the source to print the route
```

Flow of Execution

START -> Read network -> Choose objective -> Backward DP -> Print route -> Update or process another batch -> END.

6. Example

For the saved four-city example, stages are [1, 2, 2, 3] and fuel price is 100 INR/litre. The fastest route 1 -> 2 -> 4 takes 10 minutes, uses 4 litres and costs 400 INR. The cheapest route 1 -> 3 -> 4 takes 16 minutes, uses 2 litres and costs 200 INR.

Conclusion

Dynamic programming finds an optimal route for the selected objective in a staged network. Adjacency lists support efficient processing, while manual road updates and batch requests allow repeated planning with current conditions.